

Reviewer Pack — Additive Energy of Sipser Function Bounds ACC⁰ Circuit Size

Entry 50301ad64fa2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-29 10:45:25 UTC

Additive Energy of Sipser Function Bounds ACC⁰ Circuit Size

Entry ID: 50301ad64fa2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-29 10:45:25 UTC

1. Conjecture

Field A (mathematical branch): Additive Combinatorics **Field B** (complexity object): ACC⁰ Circuit Complexity

Statement:

For the Sipser function on n variables, its truth-table additive energy E is $\Omega(n^2)$, implying that any ACC⁰ circuit computing it requires size $\Omega(n^2/\varepsilon^2)$ for any $\varepsilon > 0$.

Rationale (proposer's reasoning):

Additive energy captures the number of structured quadruples (a,b,c,d) with $a+b=c+d$ in the truth-table. High additive energy indicates non-ACC⁰-computability due to the lack of low-depth linear threshold gates to handle such structured correlations, aligning with Williams' 2014 methods for NEXP vs ACC⁰.

Taxonomy category: ACC_SIPSER (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 03a756048ba63e6a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.0s

5.1 Generated Python source

```
import random
import math

def sipser_function(n, i):
    return sum(1 if (i & (1 << j)) else 0 for j in range(n) if i % (2 * (j + 1)) == 0)

def additive_energy(n):
    E = 0
    for a in range(2**n):
        for b in range(2**n):
            for c in range(2**n):
                for d in range(2**n):
                    if sipser_function(n, a) + sipser_function(n, b) == sipser_function(n, c) + sipser_function(n, d):
                        E += 1
    return E

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    E = additive_energy(n)
    expected_E = n**2
    metric_value = E / expected_E if expected_E != 0 else float('inf')
    conjecture_holds = abs(metric_value - 1) < 0.1
    counterexample = "" if conjecture_holds else f"n={n}, E={E}, expected_E={expected_E}"
    return {
        "metric_name": "Additive Energy Ratio",
        "metric_value": metric_value,
        "instances_tested": n**4,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)] # Default to first 8 primes
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
```

```

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Additive energy does not scale quadratically\"")
else:
    print("RESULT: INCONCLUSIVE insufficient data or conflicting results")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Extend timeout duration and re-run test with increased computational resources

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	62653
2	preregistration	ollama_remote	qwen3:8b	0	0	24162
3	novelty	ollama_remote	qwen3:8b	0	0	20789
4	novelty	ollama_remote	qwen3:8b	0	0	14160
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29955
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7601
7	judge	ollama_remote	qwen3:8b	0	0	17124

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 176443 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in [research/audit/50301ad64fa2.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/50301ad64fa2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/50301ad64fa2.tar.gz` (if generated)